

Infosys Springboard Virtual Internship 7.0 Completion Report

Team Details

Batch Number – Team A

Start date: 21 July 2026

Names:

Abhigyan Saini

Azmeera Raghavendar

Ankitha KS

Esha Yadav

Internship Duration: 8 Weeks

1. Project Title

AI-Based Industrial Maintenance and Process Optimization System

2. Project Objective

The main objective of the **AI-Based Industrial Maintenance and Process Optimization System** is to develop an intelligent platform that monitors industrial machines, analyzes operational data, and predicts potential machine failures in advance.

The system aims to:

- Monitor machine health using **real-time and historical sensor data**.
- Analyze parameters such as **temperature, vibration, pressure, RPM, humidity, voltage/current, oil level, and load**.
- Use **machine learning techniques** to identify abnormal machine conditions and predict potential failures.
- Classify machine conditions into **Normal, Possible Failure, and High Risk**.
- Provide a centralized dashboard for **machine monitoring, telemetry, maintenance, incidents, inventory, and notifications**.
- Support **proactive maintenance and process optimization** to reduce unexpected downtime and improve machine reliability.
- Help maintenance teams make **data-driven decisions** and improve overall industrial operational efficiency.

3. Project description in detail

My internship project, RoboPulse AI: Predictive Robot Arm Monitoring and Maintenance Platform, is a web-based system developed to monitor industrial robot arms, identify potential equipment failures, calculate robot health, and support maintenance planning.

Industrial robots operate continuously in manufacturing environments. Failures in motors, joints, sensors, bearings, or electrical components can stop production and create maintenance costs. In many cases, teams detect these issues only after a fault occurs. This project addresses that problem by collecting robot telemetry data, analyzing it with machine learning, and presenting the results through a centralized dashboard.

The platform monitors important robot parameters such as joint temperature, joint current, joint speed, tool current, and operating cycle information. These values help indicate whether a robot is running normally or showing early signs of stress. The system stores the data and allows users to review it through tables, charts, and robot-specific views.

Approach Used

The project follows a predictive-maintenance approach.

First, robot-related data is stored in the system. This includes robot details, sensors, telemetry records, maintenance records, incidents, notifications, and users.

Second, the latest telemetry data for a selected robot is passed to a machine-learning model. The model analyzes 20 features, including current and temperature readings from multiple joints, joint speeds, tool current, and cycle data. It estimates the probability that the robot may experience a protective stop or failure.

Third, the system presents a clear diagnosis to the operator. It displays the predicted fault, failure probability, and a maintenance recommendation. The result is also saved in the historical prediction log for future review.

The platform also calculates a health score for each robot. The health score uses all joint temperatures, average motor current, average joint speed, tool current/load, and machine-learning failure risk. The score is shown on a scale from 0 to 100 and classified as Excellent, Good, Warning, Critical, or Failure Likely.

Finally, if a robot shows a high risk of failure, the operator can create a maintenance record, assign a technician, set a due date, log an incident, or send a notification.

Technologies Used

- Python for backend development and machine-learning integration
- Streamlit for the interactive web dashboard
- FastAPI for REST API development
- SQLite for storing robot, telemetry, prediction, incident, and maintenance data
- SQLAlchemy for database models and queries
- Pandas for telemetry data processing
- Scikit-learn for the Random Forest machine-learning model
- Joblib for loading the trained model

- HTML and CSS for dashboard styling and navigation design
- Token-based authentication for protected user access

Key Features

- Fleet overview for monitoring multiple robot arms
- Robot asset and sensor management
- Telemetry storage, charts, and historical data review
- Automated ML-based failure-risk prediction
- Interactive ML simulator for testing telemetry inputs
- Historical prediction logs
- Health diagnostics with a 0–100 health score
- Maintenance scheduling and technician assignment
- Incident tracking and severity management
- Alert and notification management
- User login and protected API access

Real-World Impact

In a real industrial environment, RoboPulse AI can support predictive maintenance by helping teams identify warning signs before a complete robot failure occurs. For example, a rise in joint temperature, current draw, or tool load may indicate wear, friction, overload, or a developing motor issue. The system can flag that robot for inspection before it causes an unexpected production stop.

The platform can help maintenance teams prioritize high-risk robots instead of inspecting every machine manually. It also creates a structured record of maintenance activities, incidents, predictions, and alerts. This improves traceability and helps supervisors understand recurring equipment problems.

With live IoT sensors and cloud deployment, the project could be extended for real factory use. Future versions could send mobile alerts, integrate with enterprise maintenance systems, support multiple factory locations, and use more advanced models trained on larger industrial datasets.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Understand the problem statement, project requirements, predictive maintenance concepts, and existing industrial maintenance systems.	Studied predictive maintenance and industrial machine monitoring. Finalized the project title, objectives, requirements, and overall system concept.
Week 2	Analyze system requirements and design the overall architecture, database structure, and application modules.	Designed the system architecture and identified the major modules: Dashboard, Robots, Sensors, Telemetry, Maintenance, Incidents, Inventory, Notifications, and Prediction.
Week 3	Develop the backend services and database for storing machine, telemetry, maintenance, and incident information.	Implemented the FastAPI backend and SQLite database. Created REST API endpoints and database models for machine monitoring and maintenance operations.
Week 4	Develop the frontend dashboard and machine monitoring interfaces.	Developed the Streamlit frontend with dashboard, robot/machine monitoring, sensor, telemetry, maintenance, and incident interfaces.
Week 5	Collect and prepare machine telemetry data and develop the machine learning prediction module.	Prepared telemetry data containing parameters such as temperature, vibration, pressure, RPM, humidity, voltage/current, and load. Developed and trained machine learning models for risk prediction.
Week 6	Integrate the trained ML model with the application and implement risk classification and maintenance features.	Integrated the Random Forest model with the system. Implemented machine risk classification into Normal, Possible Failure, and High Risk and connected prediction results with the dashboard.
Week 7	Test the complete system, identify errors, and improve system performance and usability.	Tested frontend, backend APIs, database operations, prediction module, maintenance records, incidents, inventory, and notifications. Fixed identified issues and improved the user interface.
Week 8	Perform final validation, document the project, prepare screenshot.	Completed system validation and documentation. Prepared project report, implement.

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Finalized the project topic, objectives, requirements, and initial plan for developing the AI-based industrial predictive maintenance system.	30 July 2026
Prototype/First Draft	Developed the initial prototype with the basic frontend, backend, database, machine monitoring, and telemetry modules.	10 August 2026
Mid-Term Review	Demonstrated the developed modules, reviewed system progress, tested the initial prediction model, and incorporated feedback.	21 August 2026
Final Submission	Completed the integrated system including machine monitoring, telemetry analysis, maintenance management, incident tracking, inventory, notifications, and AI-based risk prediction.	4 September 2026
Presentation	Prepared and presented the final project demonstration, results, system architecture, future scope, and conclusion.	16 September 2026

5b. Project execution details

The RoboPulse AI project was executed using a modular approach, with the system divided into frontend, backend, database, machine learning, and maintenance-management components. The development was carried out in multiple stages, starting from requirement analysis and system design and progressing toward implementation and integration.

First, the requirements and system architecture were defined. The main objective was to develop a centralized platform for monitoring industrial robotic arms, collecting telemetry data, analysing machine conditions, and supporting predictive maintenance. The system was planned around the flow of robot and sensor data → FastAPI backend → database and machine-learning model → Streamlit dashboard.

The backend was developed using FastAPI. Separate API routers were created for robotic arms, sensors, telemetry, predictions, maintenance, incidents, notifications, users, and health diagnostics. This modular structure helped keep different functionalities organized and made the application easier to maintain and extend. REST APIs were used to allow communication between the frontend and backend.

For data storage, SQLite was used as the database and SQLAlchemy ORM was implemented for database operations. Models were created for entities such as RobotArm, Sensor, Telemetry,

Prediction, Maintenance, Incident, Notification, and User. The telemetry model stores important machine parameters including temperature, vibration, motor current, voltage, power consumption, torque, speed, operating hours, humidity, and timestamp. Prediction records store the failure probability, predicted fault, confidence, recommendation, and prediction time. Pydantic schemas were also used to validate the data exchanged through the APIs.

The next stage involved implementing telemetry and predictive analytics. Machine-condition data is collected and stored through the telemetry APIs. The collected parameters are then used by the machine-learning component to analyze the condition of the robotic arm. A Random Forest classifier was integrated into the system to generate predictive results. The model produces outputs such as failure probability, predicted fault, confidence, and recommended action. The system categorizes the predicted risk into different levels, with higher failure probabilities indicating a greater risk of failure.

The Streamlit frontend was then developed to provide an interactive dashboard for operators. It communicates with the FastAPI backend through REST requests and displays information retrieved from the database and prediction services. The dashboard provides sections for fleet overview, robotic assets, sensor information, telemetry, predictive analytics, maintenance scheduling, incident tracking, notifications, health diagnostics, and user management. Important metrics such as the total number of robots, active robots, warning conditions, and critical conditions can be displayed to provide a quick overview of the robotic fleet.

Authentication and user management were also incorporated into the application. User session information and access tokens are maintained by the frontend, and authenticated requests are sent to the backend using token-based authorization. This provides controlled access to the application's APIs and dashboard features.

The project was finally integrated and tested as an end-to-end system. FastAPI's Swagger UI was used to test and verify the API endpoints, while the Streamlit application was used to verify the complete user workflow. The final flow of the system is:

Robotic/Sensor Data → Telemetry API → Data Validation → Database → Random Forest Prediction → Failure Probability and Fault → FastAPI Response → Streamlit Dashboard → Maintenance/Operational Action.

Thus, the project execution involved integrating Python, FastAPI, Streamlit, SQLite, SQLAlchemy, Pydantic, Pandas, and Random Forest machine learning into a single platform. This approach resulted in a system capable of combining robotic-arm monitoring, telemetry management, predictive analytics, and maintenance-related activities within one centralized application.

6. Snapshots / Screenshots

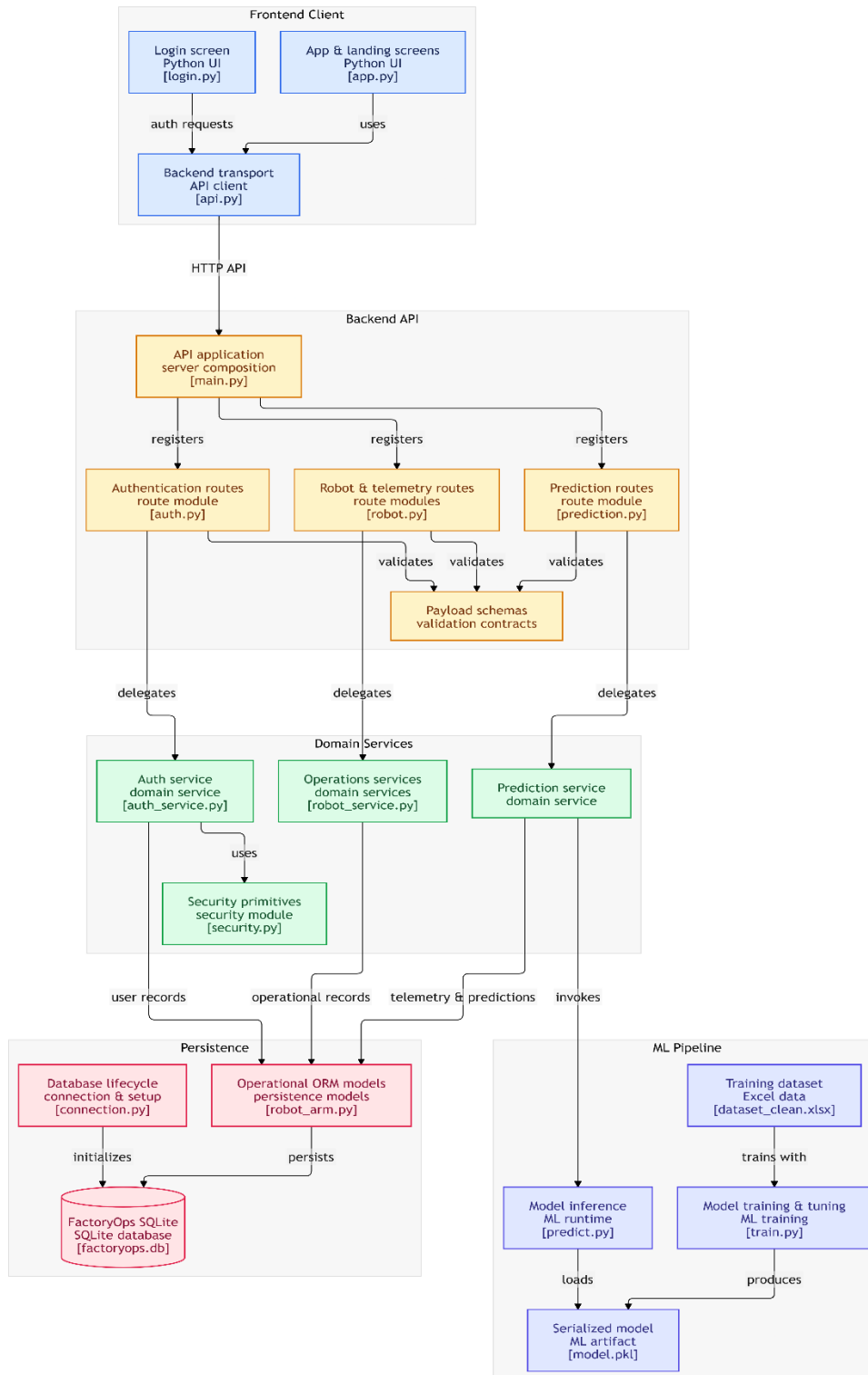


Figure 1: RoboPulse AI System Architecture

```
app = FastAPI(  
    title="Predictive Robot Arm API",  
    version="1.0.0",  
    description="Predictive Maintenance API for Industrial Robot Arms"  
)
```

Figure 2: FastAPI backend configuration and modular API router implementation.

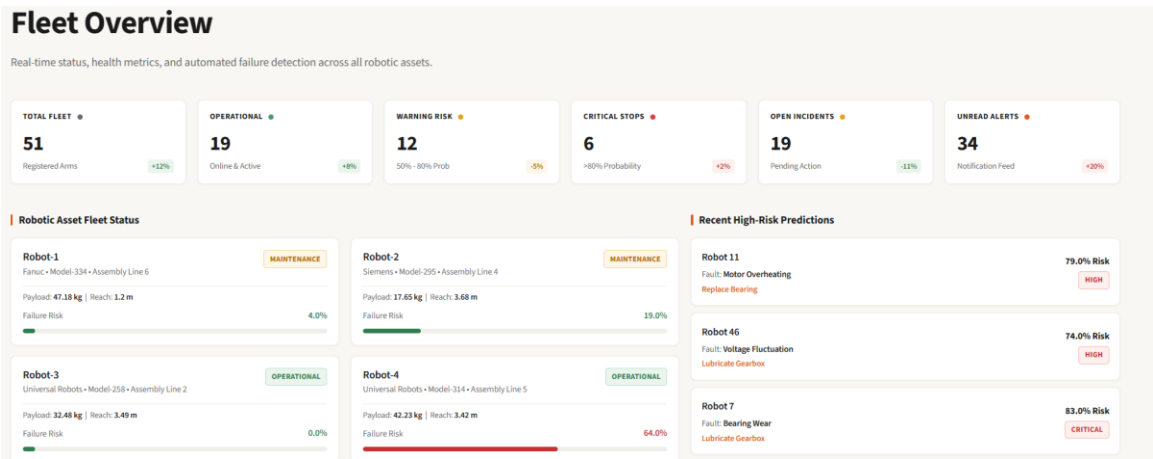


Figure 3: Centralized RoboPulse AI dashboard for robotic fleet monitoring

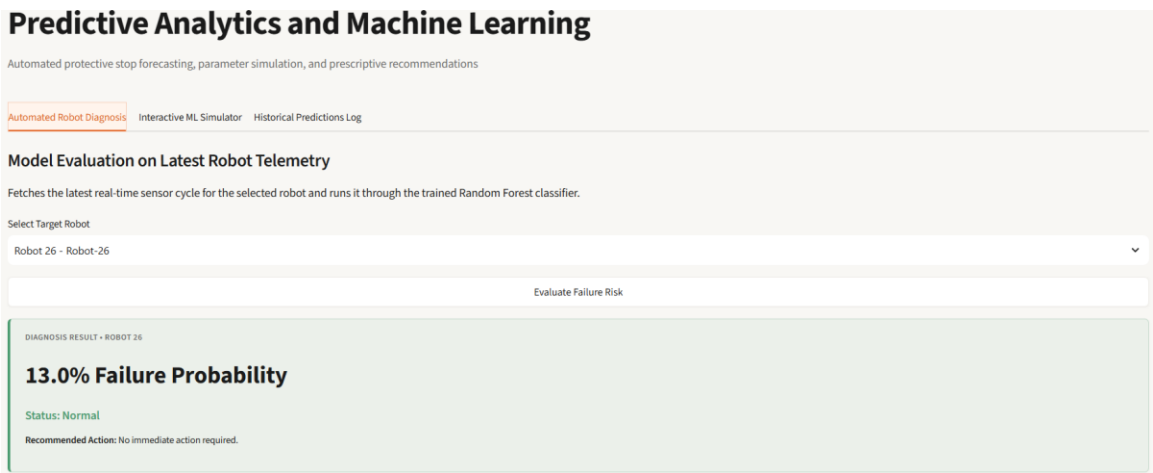


Figure 4: Random Forest-based predictive analytics for identifying potential robotic-arm failures.

Health Diagnostic Station

Compute real-time health scores from all joint temperatures, currents, speeds, tool load, and ML failure risk

Select Robot Arm

Robot 51 - xyzlm

Calculate Health Score

ROBOT 51 HEALTH DIAGNOSIS

71 / 100

Condition: Warning

Thermal Reading

45.8 °C

Average Joint Speed

135.00

Motor Current

1.79 A

Tool Current / Load

0.5 A

Figure 5: Health diagnostic interface for evaluating the condition of robotic arms

Joint Telemetry and Waveform Analytics

High-frequency joint currents, thermal curves, speeds (J0-J5), and tool cycle metrics

> Ingest Field Telemetry Data Point

Select Target Robot

All Robots

Showing the latest 300 of 7410 records for smooth performance. Use the Raw Feed tab to export all data.

Currents (J0-J5) Thermal Profiles (T0-J5) Joint Speeds (J0-J5) Raw Telemetry Feed

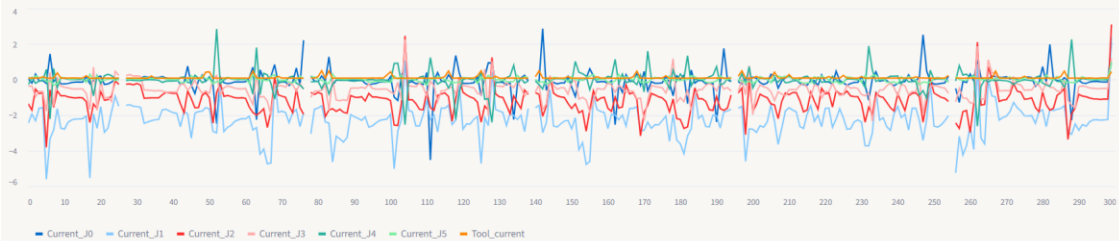


Figure 6: Telemetry Dashboard

7. Challenges Faced

A major technical challenge was integrating the FastAPI backend with the Streamlit frontend and ensuring proper communication between the different modules. This was resolved by testing APIs individually using Swagger UI and validating the responses before integrating them with the dashboard.

Another challenge was managing the SQLite database and SQLAlchemy relationships, especially while handling telemetry, robot, and prediction records. These issues were addressed by reviewing the database models, relationships, and constraints and testing database operations separately.

Integrating the Random Forest machine-learning model with the application was also challenging, particularly in handling telemetry inputs and displaying prediction results correctly. This was managed by validating the input features and prediction outputs before connecting them to the dashboard.

From an operational and communication perspective, understanding requirements and coordinating changes across different modules required regular discussions and feedback. These interactions helped clarify requirements and resolve implementation issues effectively. Overall, the challenges provided valuable practical experience in debugging, API integration, database management, machine-learning implementation, and teamwork.

8. Learnings & Skills Acquired

The internship provided valuable hands-on experience in developing a real-world predictive maintenance system. I gained practical knowledge of Python, FastAPI, Streamlit, SQLAlchemy, SQLite, REST APIs, Pandas, and Random Forest. I also developed an understanding of robotic-arm monitoring, telemetry data, predictive maintenance, and machine-learning-based failure prediction.

The project improved my ability to integrate frontend, backend, database, and ML components into a single application. In addition, I strengthened my problem-solving, debugging, communication, teamwork, and time-management skills through regular development and collaboration. Overall, the internship helped me apply academic concepts to a practical industrial application and understand the complete software development workflow.

Additionally, the internship helped me understand the importance of testing, documentation, and continuous improvement in software projects. I learned how to identify technical issues, analyze their causes, and implement suitable solutions. Working on the project also gave me greater confidence in handling end-to-end application development and applying technical knowledge to solve practical industry problems.

9. Testimonials from team

The internship was a valuable learning experience that provided me with the opportunity to work on a real-world predictive maintenance project for industrial robotic arms. I successfully contributed to the development of RoboPulse AI, integrating the frontend, backend, database, and machine-learning components into a single application.

A key achievement was gaining hands-on experience with FastAPI, Streamlit, SQLAlchemy, SQLite, REST APIs, and Random Forest. I also successfully implemented features for robot monitoring, telemetry management, predictive analytics, health diagnostics, and maintenance management.

The internship also strengthened my problem-solving, debugging, teamwork, communication, and time-management skills. Successfully resolving technical issues and integrating different modules gave me confidence in developing practical software solutions and applying my academic knowledge to an industry-oriented project.

10. Conclusion

The internship was a valuable experience that helped me gain practical exposure to software development and machine learning. Working on the RoboPulse AI project allowed me to apply academic concepts in a real-world industrial application involving robotic-arm monitoring and predictive maintenance.

The internship improved my technical knowledge of Python, FastAPI, Streamlit, SQLAlchemy, SQLite, REST APIs, and Random Forest while also strengthening my problem-solving, debugging, communication, and teamwork skills. It gave me a better understanding of how different technologies are integrated to develop a complete application.

This experience aligns well with my Computer Science academic background and career goals, particularly in areas such as software development, backend development, machine learning, and intelligent industrial systems. It also provided a strong foundation for applying my academic knowledge to future industry-oriented projects.

11. Acknowledgements

I would like to sincerely thank INFOSYS for providing me with the opportunity to undertake this internship and gain valuable practical experience in a professional environment. I am grateful to my mentor, Selciya for providing continuous guidance, valuable feedback, and support throughout the project.

I would also like to thank all the team members and colleagues who assisted me during the internship, shared their knowledge, and helped me overcome technical challenges. Their support and encouragement played an important role in completing the RoboPulse AI project successfully.

Finally, I am thankful to everyone who contributed directly or indirectly to making this internship a meaningful and enriching learning experience.